EPFL · COM-480 · DATA VISUALIZATION · FINAL MILESTONE · SPRING 2026

Portfolio Allocation Engine

From raw prices to optimal weights : a story in three acts

Team: *Petrit Arifi* · *Adam Ait Bousselham* · *Florian Dinant* **Repo:** click here **Demo:** click here **Video:** <2-min-link>

1 Project at a glance

Modern Portfolio Theory (MPT) is taught with three or four equations on a whiteboard and a single static frontier curve. Yet behind that curve sits an extraordinarily rich object: a high-dimensional surface shaped by a noisy covariance matrix, deformed by every assumption the modeller makes about expected returns, and traversed by a handful of competing portfolio strategies (min-variance, tangency, risk-parity, mean-variance) that often disagree. Our project turns that surface *back into a thing you can poke at.*

The Portfolio Allocation Engine is a single-page web application that ingests daily prices for nine diversified assets, recomputes the entire MPT pipeline live in the browser, and renders it through fifteen interlinked, hand-built D3 visualisations spread across three thematic tabs. Everything moves when the user changes a parameter; nothing is precomputed.

Data story

We tell a three-act story. (1) *Get to know your assets* : distributions, correlations, rolling regimes. (2) *Estimate risk honestly* : three covariance estimators, side by side, and what their differences mean. (3) *Build a portfolio* : the efficient frontier, four optimal portfolios, and a 10 000-point Monte-Carlo cloud that makes the geometry of optimisation tangible.

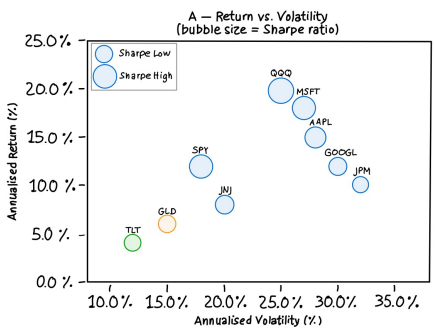
Target audience

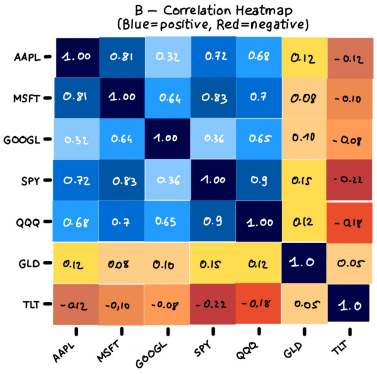
The intended reader is the curious finance-adjacent learner : undergraduate students in finance, economics or engineering; quantitative finance enthusiasts; or any retail investor who has heard the term *efficient frontier* and would like to understand where it actually comes from. We deliberately avoided the two extremes: the over-simplified "pick three stocks" robo-advisor, and the intimidating Bloomberg / Python-notebook workflow. The visualisations assume only that the user knows what *return* and *risk* are.

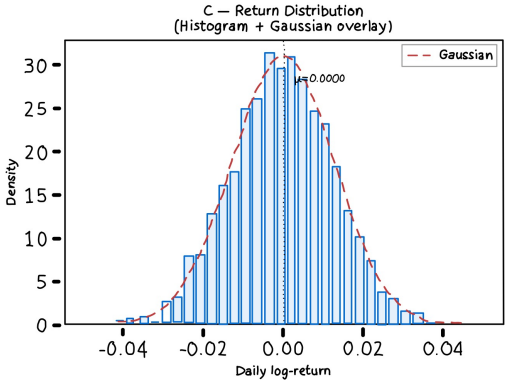
2 From milestone 2 sketches to the final product

2.1 The initial idea

Our milestone-2 proposal centred on *visualising the efficient frontier as an interactive Markowitz playground.* The original sketches contained six panels : a returns vs volatility graph, a static correlation heatmap, a return distribution for each asset, a covariance estimator comparator, the efficient-frontier itself, and portfolio weights comparator.







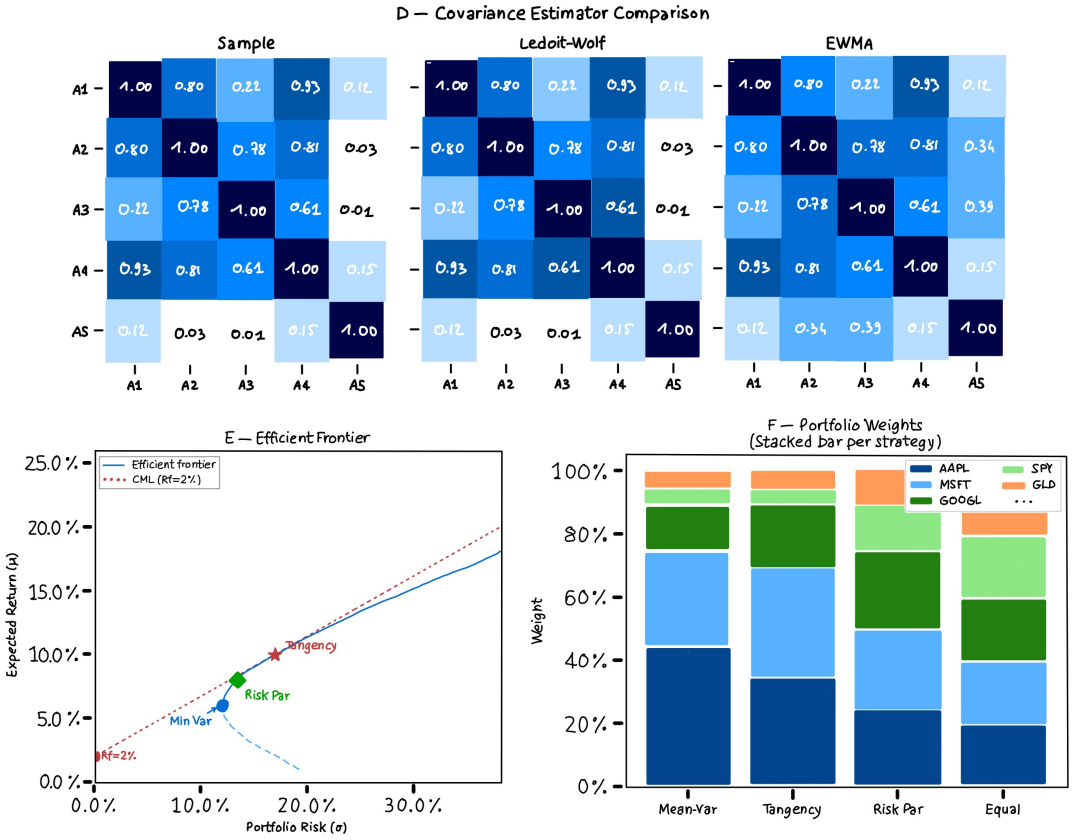


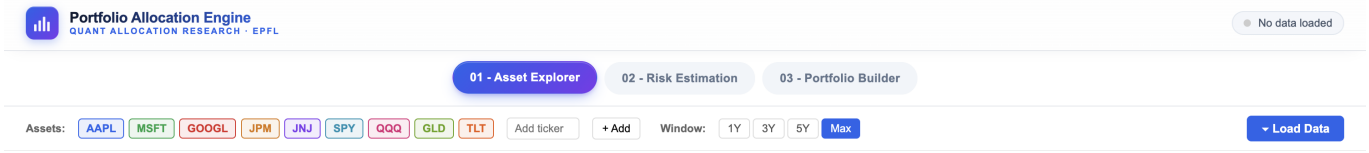
Figure 1: Original milestone 2 sketches

Two things bothered us once we tried to prototype it.

1. **The frontier alone is not a story.** It is the *conclusion* of a long pipeline that the user never sees. Showing only the conclusion makes the visualisation poor and we wanted to focus not only on Markowitz theory.
2. **Static covariance is a silent assumption.** Markowitz textbook treatment fixes $\hat{\Sigma}$ as the sample covariance, which is well known to be unstable in finite samples. Pretending the choice does not matter would have been intellectually dishonest, given the audience.

2.2 The pivot to a three-act structure

We restructured the application around a short pedagogical narrative: **Explore** → **Estimate** → **Optimize**. Each act became a tab. The tabs share state - the same chip-list of active assets and the same look-back window drive every panel - so that toggling an asset or shortening the window propagates instantly across all three tabs.



2.3 What we kept, what we changed

Kept	The efficient frontier as the structural climax of the app; the correlation heatmap; the asset-by-asset summary table.
Refined	The frontier became annotated with four named portfolios and now lives next to a Monte-Carlo cloud that proves it is the upper envelope.
Added	The entire *Risk Estimation Studio* (Tab 2). Rolling statistics for time-varying regimes (Tab 1). Stacked weight bars across strategies (Tab 3). Histogram with normal fit.
Removed	A planned "backtesting" tab. We cut it because it would have shifted the focus from *how the math works* to *which strategy wins*, which is a different question.

2.4 The landing page : an editorial entry point

A late but consequential addition to the project was a dedicated *landing page* : a full-viewport hero that the user meets *before* any chart, any chip, any slider. Our milestone-2 prototype dropped the user directly onto Tab 1, which made the application feel like a notebook rather than a product, and gave first-time visitors no sense of *why* they were looking at a return-vs-volatility scatter. The hero solves that.

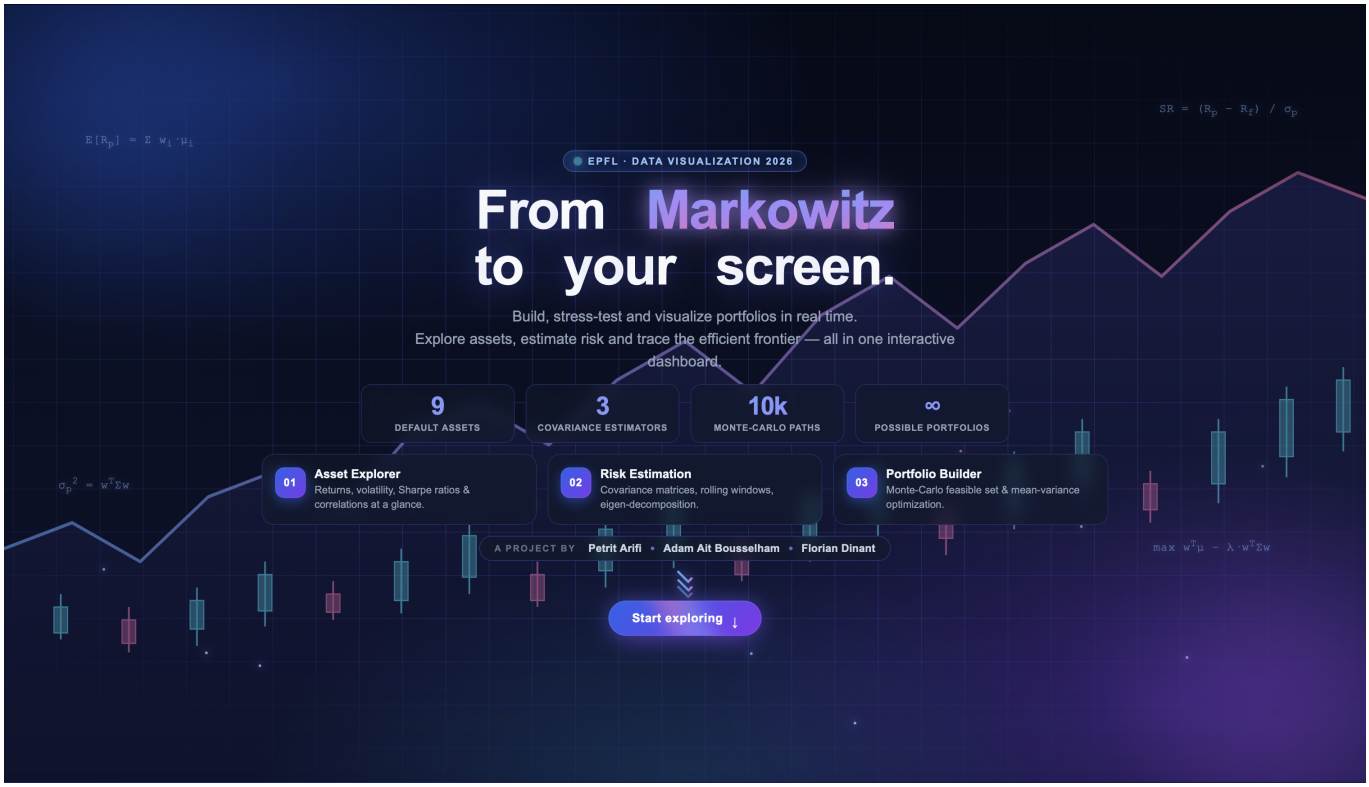


Figure 2: Landing page — the editorial entry point. The user lands on the tagline "*From Markowitz to your screen*", sees four key figures (9 assets, 3 estimators, 10 000 Monte-Carlo paths, ∞ portfolios) and the three-act structure of the app, before scrolling down to the engine itself.

What it shows. The hero is built around four nested layers, each carrying a specific informational role :

- A **tagline** (*"From Markowitz to your screen."*) that frames the project as a translation : from a 1952 paper to a 2026 browser. The two halves of the title are intentionally typeset on two lines, with *Markowitz* in a gradient fill, so the eye lands first on the historical anchor.
- A **lead paragraph** that names the three verbs of the app (*build, stress-test, visualize*) and previews the Explore $\rightarrow$ Estimate $\rightarrow$ Optimize narrative.
- Four **counted-up KPIs** (9 / 3 / 10K / ∞) that telegraph the scope of the engine without forcing the user to read.
- A **three-card preview** of Tabs 01-02-03, mirroring the tab labels exactly so that the curriculum is visible *before* the user enters it.

Visual language. The hero deliberately departs from the rest of the application. The engine itself uses a minimalist *light* editorial palette (white cards, light-grey background, a single accent blue) ; the hero is *dark*, animated, with a gradient candlestick chart, four floating LaTeX-style formulas ($E[R_p] = \Sigma w_i \mu_i$, $\sigma_p^2 = w^T \Sigma w$, the Sharpe ratio, the mean-variance objective), a scan-line sweep on boot, and a particle field. The contrast is intentional : the dark hero plays the role of a *cover page*, the light engine the role of the *document*. Crossing from one to the other (via the "Start exploring" CTA, which smoothly scrolls to the tab anchor) is a deliberate transition from *promise* to *tool*.

Why we added it. Three reasons, in order of importance. (1) *Audience priming.* Our target reader is a curious finance student, not a quant ; the hero gives them ten seconds to decide that this is for them, before

the dashboard demands attention. (2) *Narrative scaffolding.* By exposing the three-act structure upfront, we eliminate the discoverability problem we observed with peers in the milestone-2 review, who reached Tab 3 without ever opening Tab 2. (3) *Identity.* A project that lives at a public URL benefits from a recognisable cover ; the hero is what someone remembers when they share the link.

Implementation note. The hero is hand-written HTML/CSS, with no JavaScript framework : the title is split into per-character `<span>`s and animated in via staggered CSS `@keyframes` ; the KPI counters use a single `requestAnimationFrame` loop that runs once on page load ; the candlestick SVG is fully static and adds zero runtime cost. The CTA's `enterEngine()` handler performs a smooth scroll to the tab anchor and triggers the first `computeAndRender()` call, so that the engine is already warm by the time the user reaches it.

3 Design decisions

3.1 Three tabs, one narrative

The three-tab structure is the most consequential design decision of the project. It enforces a reading order : the user *must* pass through Tab 1 before the labels in Tab 2 (covariance estimators) make sense, and through Tab 2 before the dropdown in Tab 3 has any meaning. The tabs therefore do double duty: they are a layout primitive *and* a curriculum.

3.2 Encoding choices

Per-asset colour identity. Each ticker is assigned a colour once, in a fixed 15-step categorical palette, and that colour is re-used everywhere: chip border, scatter bubble, cumulative line, heatmap label, weight stack. The user never has to relearn a legend.

Figure 3: Per-asset colour identity

Bubble size = Sharpe. On the return-vs-volatility scatter, the bubble's *position* encodes the textbook risk/return pair, but the *size* encodes Sharpe. A high-Sharpe asset is therefore visible at a glance even before the user reads the axes. We considered encoding market cap or volume in the size channel; we rejected both because they tell a different story (size of company, not quality of investment) and would have competed with the chart's purpose.

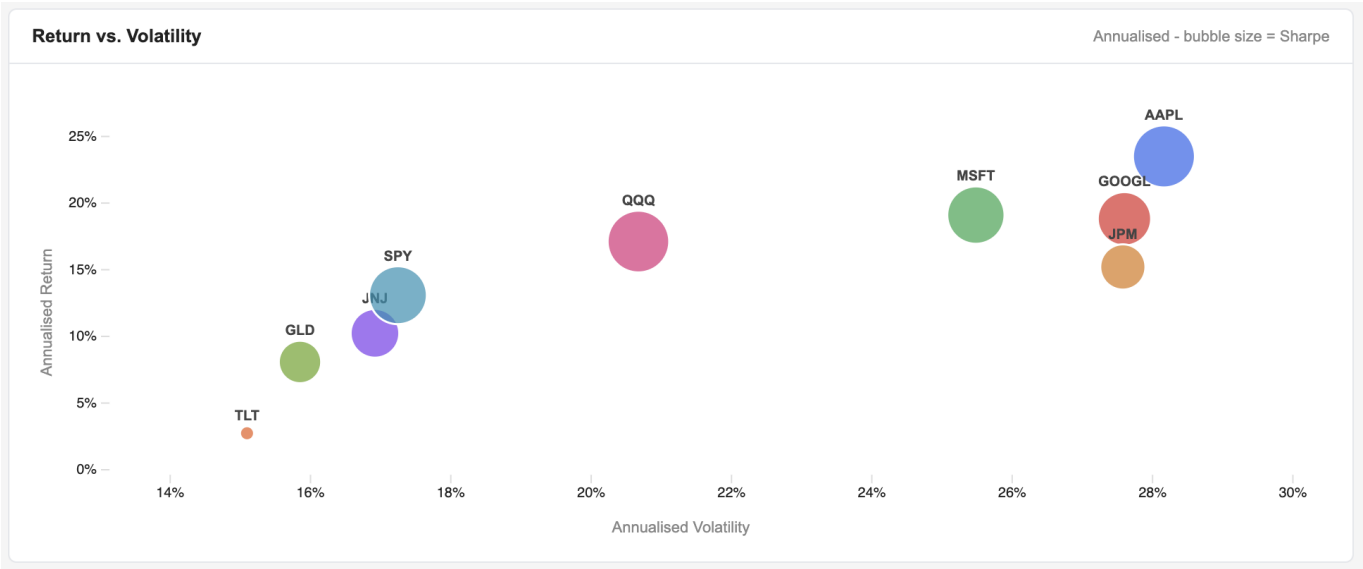


Figure 4: Per-asset colour identity

Divergent vs. sequential heatmaps. The correlation heatmap in Tab 1 uses `interpolateRdBu` on $[-1, 1]$

because 0 has a genuine semantic meaning (independence). The covariance heatmaps in Tab 2, by default, use `interpolateYlOrRd` on [min, max] because 0 is no longer a meaningful midpoint once we annualise covariance. Toggling *"Show difference vs. Sample"* switches the LW and EWMA panels back to a divergent palette because the sign of the difference is now informative.

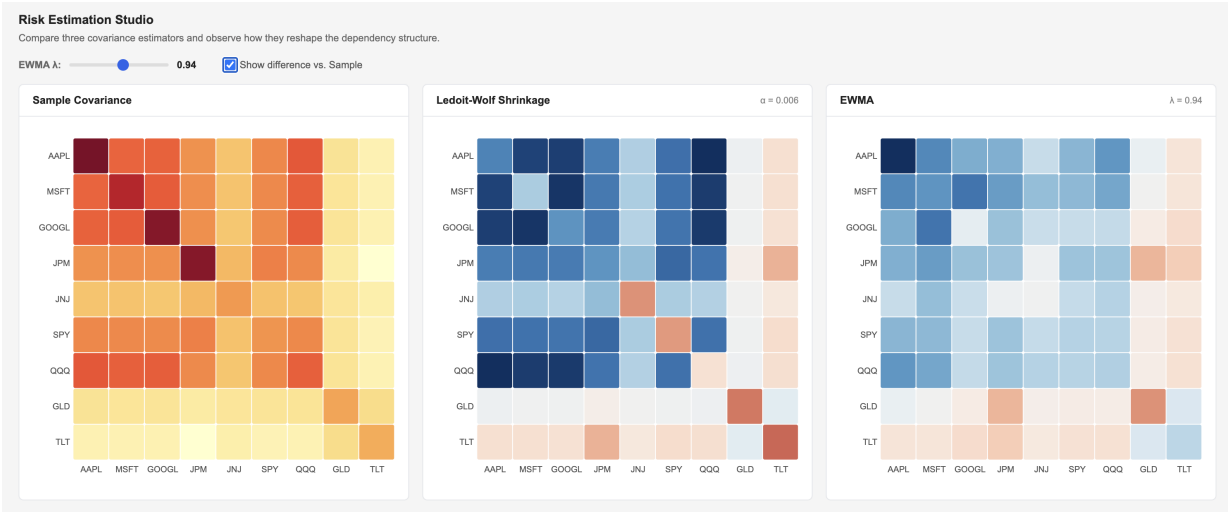


Figure 5: Sample covariance / Ledoit-Wolf / EWMA, with diff toggle on.

3.3 Animation as pedagogy

We use animation deliberately and sparingly. Each transition is a *teaching moment*, not decoration.

- **Scatter bubbles grow in** from $r = 0$ on a 40 ms cascade. The cascade lets the user see *which asset enters last* - visually flagging outliers in volatility space.
- **Monte-Carlo cloud paints in over 2.6 s** with a cubic-out easing, so the user can watch the feasible set fill out and realise that the efficient frontier is its *upper-left envelope*. Without the animation, the cloud is just a blob.
- **The pulsing star** on the running-best portfolio is recomputed at every frame. It jumps around violently in the first 200 ms and then locks onto a stable point, dramatising the law of large numbers in front of the user's eyes.
- **The frontier draws itself in** via a stroke-dasharray tween of length $\to 0$, fading in only after the cloud is complete. The temporal ordering tells the user *the frontier is a derived object.*

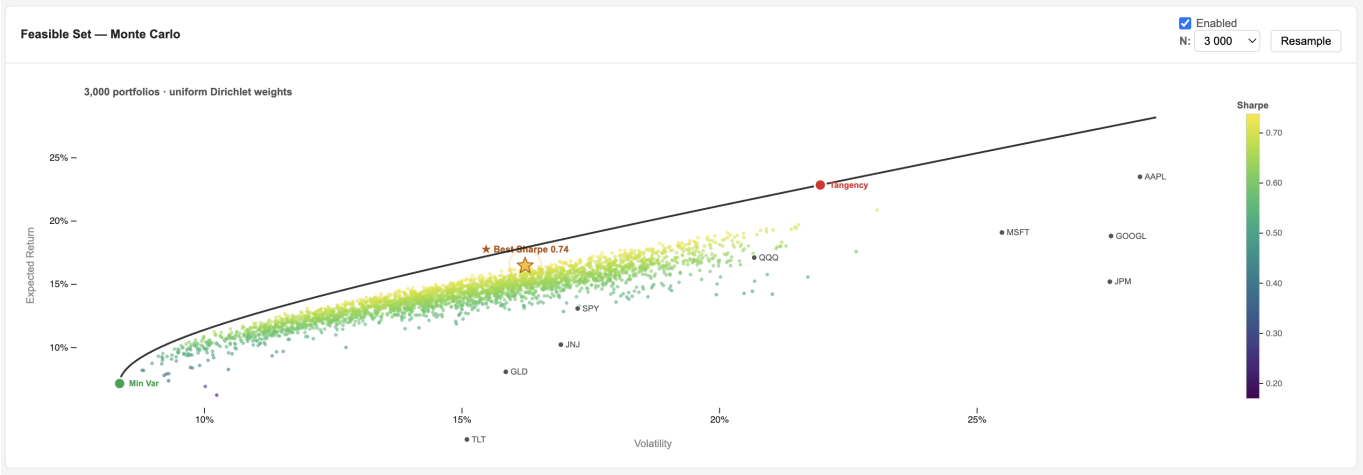


Figure 6: Feasible Set — Monte Carlo

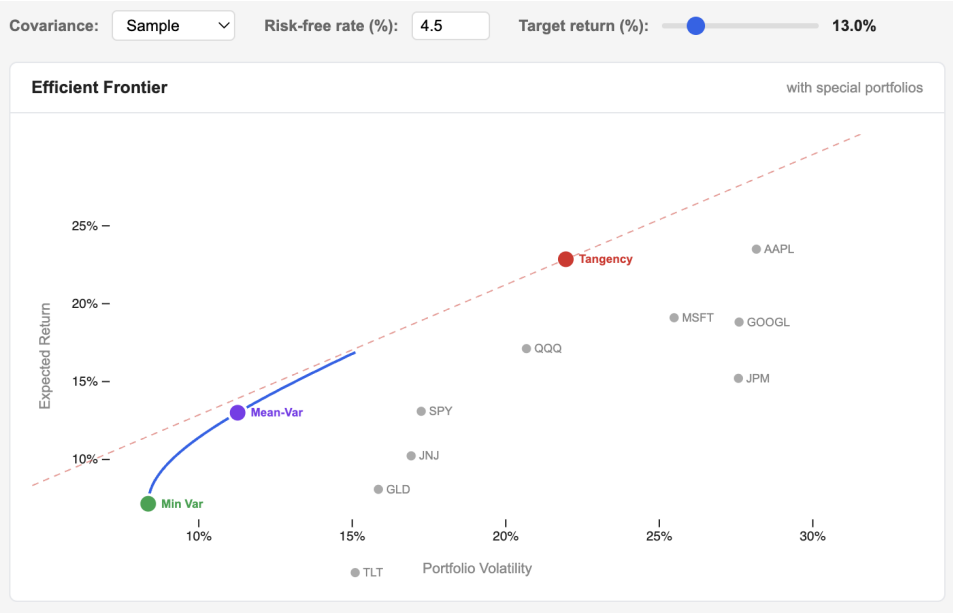


Figure 7: Efficient Frontier

3.4 Layout and typography

We adopted a minimalist editorial palette: white cards on a light-grey background, a 13 px Inter (with Arial fallback) for body, and a single accent blue (`#2563EB`) for affordances. Numerical columns use tabular figures (`font-variant-numeric: tabular-nums`) so that comparisons read down a column without optical drift. Negative weights (shorts) in the stacked-weights chart are rendered with a dashed red border, never a solid fill, to encode their exceptional status.

4 Technical implementation

4.1 Stack

The application is intentionally stack-light: a single `index.html`, a single `style.css`, a single `engine.js`, and one CSV. **No build step. No bundler. No framework. No charting library beyond D3.js.** It runs on any HTTP server and on GitHub Pages.

4.2 Math, from scratch

A deliberate constraint was to write every numerical routine ourselves, so that the visualisations remain explainable line by line.

Routine	Implementation
Matrix inversion	Gauss-Jordan with partial pivoting (`matInv`)
Eigendecomposition	Jacobi rotations on the symmetric covariance, $\mathcal{O}(n^3)$ per sweep, used for the eigenvalue spectrum and condition number
Sample covariance	Centred outer product divided by $T-1$
Ledoit-Wolf shrinkage	Asymptotically optimal $\alpha = \min(\beta^2/\delta^2, 1)$ shrunk towards a scaled identity
EWMA covariance	Exponential weights λ^{T-1-k}, normalised by their sum
Markowitz frontier	Closed-form Lagrangian, $w^\star = \Sigma^{-1}(\lambda_1 \mathbf{1} + \lambda_2 \mu)$, swept across 80 target returns
Min-variance, Tangency	Closed-form, derived from the inverse-covariance row sums
Risk parity	Fixed-point iteration on equal risk contribution, 500 steps, converges in ~ 50 for our data
Dirichlet sampling	Gamma trick: $u_i = -\log(1-U_i)$ then normalise

4.3 Performance: SVG where it counts, Canvas where it scales

The Monte-Carlo panel renders up to 10 000 portfolios. With SVG, this caused 200–400 ms layout stalls every time the user moved the EWMA slider. We adopted a hybrid strategy: the static frame, the axes, the

frontier path, the special-portfolio markers and the hover ring are all SVG (crisp, exportable, accessible); the cloud itself is rendered to a *layered* `<canvas>` with the same coordinate system. This brings the per-frame cost to under 4 ms and lets us animate the 2.6 s paint-in smoothly. Hover detection is implemented in JS over an invisible SVG `<rect>` that shadows the entire chart area - nearest-neighbour search across 10 000 points takes under 0.5 ms.

4.4 State pipeline

A single function, `computeAndRender`, owns the entire pipeline. Whenever the user changes anything (asset chip, look-back window, EWMA λ, target return, risk-free rate, covariance estimator), the same five-stage pipeline is re-run:

filterPrices → **computeStats** → **computeCovariances** → **computePortfolios** → **render***

This pipeline-on-every-change approach is the simplest possible mental model for the user: *everything you see is always consistent with the parameters above it.* We measured the full pipeline at $\sim$ 180 ms for the default 9-asset / 3-year window on a 2021 MacBook Air, well within the 100 ms-per-interaction heuristic for interactive responsiveness.

5 Challenges

5.1 Singular and near-singular covariance matrices

When the user toggles assets aggressively or chooses an extremely short window (e.g. 1 Y with two highly-correlated tech tickers), the sample covariance becomes near-singular and the closed-form optimiser produces numerically explosive weights. We addressed this in three ways:

1. Our Gauss-Jordan inverter returns `null` on a pivot smaller than 10^{-14}, and downstream renderers display a graceful empty-state instead of crashing.
2. The Risk-Estimation Studio surfaces this exact pathology: a high condition number in the table is the *visual warning* that the matrix is ill-conditioned. Switching to Ledoit-Wolf or EWMA brings the condition number down by 1–2 orders of magnitude.
3. For the Mean-Variance target portfolio, we accept the possibility of negative weights (shorts), and we show them explicitly with the dashed-red encoding in the stacked bars, rather than hiding them behind a constraint.

5.2 The animation budget

We had to balance two competing goods. (a) The 2.6 s Monte-Carlo animation is critical for first-time users - it is the visualisation that converts "a frontier" into "the upper-left envelope of a feasible set". (b) For a returning user who has already seen the animation and is now moving the EWMA slider rapidly, replaying 2.6 s every time would be infuriating. We solved this with a cache keyed on `N|covariance|window|tickers|seed`: identical parameter sets skip the animation and paint the cloud instantly, while genuinely new parameters re-trigger the paint-in. A `Resample` button forces a new draw on demand.

5.3 Cross-tab consistency

A subtle bug haunted us for several days: changing the EWMA λ in Tab 2 did not propagate to the optimiser in Tab 3 if the covariance dropdown in Tab 3 was set to *EWMA*. The fix was architectural - we now recompute the EWMA covariance inside `renderPortfolio`, not just inside `renderRisk`, so the two tabs draw from the same up-to-date matrix. The bug was a useful reminder that *shared interactive state* is the hardest part of a multi-panel visualisation; it is much easier to sketch than to maintain.

5.4 Pedagogical pacing

The hardest non-technical decision was: *how much to reveal, when?* A user can in principle change the covariance estimator *before* they understand what a covariance estimator is. We debated locking the tabs in order, gating Tab 2 until Tab 1 had been visited at least once. We rejected the idea: respecting the user's agency is a load-bearing principle of the design. Instead, we made the tabs labelled `01`, `02`, `03` and we lead each tab with a one-line subtitle ("Statistical overview…", "Compare three covariance estimators…",

"Construct optimal portfolios…") so the curriculum is suggested, not enforced.

6 Conclusion

The project taught us three lessons that we will carry to every future visualisation. **First, animation is not decoration** : a well-placed 2.6-second paint-in can do the work of two paragraphs of explanation, but only if it is timed so the user can watch each phase unfold. **Second, computing live in the browser is liberating** : the entire pipeline runs in < 200 ms, which means *every* parameter can be a slider, and the user is never blocked by a back-end. **Third, a visualisation is a curriculum.** By imposing a soft narrative order : Explore, Estimate, Optimize, we turn what could have been a wall of charts into a 10-minute self-guided lesson on the geometry of Modern Portfolio Theory.

7 Peer assessment

Honest, granular breakdown of contributions per team member. The three members carried roughly equivalent loads, with the breakdown below reflecting the primary owner of each component. Where two or more members worked jointly on a deliverable, it is listed under Joint work.

Member	Main contributions
Florian	**Project foundation:** initial template, three-tab scaffold, controls bar, KPI strip, and tab-routing logic. **Core data pipeline** (`loadData` → `filterPrices` → `computeStats` → `computeCovariances` → `render*`), asset selection, and global state propagation across tabs. **Mathematical core:** Gauss–Jordan matrix inversion, Jacobi eigenvalue solver, log-return pipeline, and descriptive statistics (means, std, skewness, kurtosis), plus correlation matrix. **Asset Explorer (Tab 1):** Return vs. Volatility scatter, Summary Statistics, Correlation Matrix heatmap, Return Distribution histogram with normal fit, and Cumulative Performance ($1 growth) curves.
Petrit	**Landing page (hero):** full-viewport editorial entry point with animated candlestick chart, floating LaTeX-style formulas, per-character title animation, counted-up KPI stats, three-card tab preview, and smooth-scroll CTA bridging the dark cover to the light engine. **Risk Estimation Studio (Tab 2):** implementation of the three covariance estimators (sample, Ledoit–Wolf shrinkage with optimal α, EWMA with interactive λ); heatmap comparison system; difference visualization mode; eigenvalue spectrum analysis; condition-number and Frobenius-distance metrics. **Rolling analytics:** rolling volatility, rolling Sharpe, and rolling pairwise correlation with shared window controls (30/60/90/252 days) and anchor-asset selection.
Adam	**Portfolio Builder (Tab 3):** Monte-Carlo feasible cloud (Dirichlet sampling via Gamma trick, hybrid SVG/`<canvas>` rendering, animated transitions, hover detection, running-best indicator) and Portfolio Comparison table. **Efficient Frontier visualization system:** rendering and interaction logic. **UI/UX system:** animation framework, cascading transitions, design system (typography, colors, components), and cross-tab interaction consistency.

Joint work: the **Efficient Frontier pipeline**, **core statistical engine integration**, and **Stacked-Bar portfolio comparison** were built collaboratively. The overall architecture (*Explore* → *Estimate* → *Optimize*), UI/UX system, README, screencast, and cross-tab debugging were handled jointly by all members.

Repository: click here **Live demo:** click here **Screencast:** click here